Project Proposal: Design and Verification of a 5-Stage Pipelined RISC-V (RV32I) CPU in VHDL

Author: Chadha Ben Said, Computer Engineering, ENSI **Date:** October 28, 2026
Supervisor: _____ **Status:** Proposal, for supervisor review and feedback

1. Context and Motivation

FPGAs and digital design are used across telecom, defense, aerospace, automotive and data acquisition. My background so far is in embedded systems, IoT and applied AI. I want to add a solid hardware-design project that shows I can write synthesizable RTL, verify it rigorously and analyze its timing and resource usage.

The project also connects directly to my coursework on advanced architectures and parallel programming (pipelining, hazards, caches, speedup), turning those concepts into a working design.

FPGA boards are expensive, so the project is designed to be **fully board-free**: everything is validated through simulation, automated verification and synthesis reports. A physical board remains an optional extension.

2. Objectives

1. Design a 32-bit RISC-V CPU (RV32I base instruction set) with a classic 5-stage pipeline (Fetch, Decode, Execute, Memory, Write-back) in VHDL.
2. Handle pipeline hazards correctly: data forwarding, load-use stalls, and control hazards (branches and jumps).
3. Verify the design with self-checking testbenches and by running real C programs compiled with the RISC-V GCC toolchain, compared against a reference model.
4. Synthesize the design for a real FPGA family and report maximum clock frequency, resource usage and critical paths.
5. Publish a clean, documented repository with automated tests (continuous integration).

Stretch goals (if time allows): a small instruction or data cache, a UART peripheral, CPI and performance analysis against a single-cycle version, and a custom instruction or small accelerator.

3. Project Description

3.1 Architecture overview

The CPU follows the classic five-stage organization:

- **IF (Instruction Fetch):** program counter, instruction memory access, next-PC selection.
- **ID (Instruction Decode):** instruction decoder, register file (32 x 32-bit), immediate generation, hazard detection unit.
- **EX (Execute):** ALU, branch comparison, forwarding multiplexers.
- **MEM (Memory Access):** data memory read and write with byte, half-word and word access.
- **WB (Write-back):** result selection and register file write.

Pipeline registers separate the stages. A forwarding unit and a hazard detection unit handle data dependencies, and branch handling starts with a simple predict-not-taken policy with flush on taken branches.

3.2 Instruction set scope

RV32I base integer instructions: arithmetic and logic, shifts, loads and stores, branches, JAL and JALR, LUI and AUIPC. The M extension (multiply and divide) is a possible later addition.

3.3 Verification approach

Verification is a core part of the project, not an afterthought:

- **Unit tests:** self-checking testbenches for each block (ALU, register file, decoder, forwarding and hazard units).
- **Instruction-level tests:** assembly tests covering every supported instruction.
- **Program-level tests:** small C programs (for example sorting, Fibonacci, matrix multiplication) compiled with the RISC-V toolchain and loaded into simulated memory.
- **Reference comparison:** execution traces compared against a reference simulator (such as Spike) or a Python model, checking register and memory state.
- **Coverage and assertions:** instruction and hazard-scenario coverage, plus assertion-style checks on pipeline invariants.

4. Tools

All tools are free or open source, so no hardware purchase is needed.

Purpose	Tool
VHDL simulation	GHDL (alternative: ModelSim or Questa free edition)
Waveform inspection	GTKWave
Python-based testbenches	cocotb
Synthesis and timing (open source)	Yosys with the GHDL plugin, nextpnr
Synthesis and timing (vendor)	Vivado or Quartus free editions, used for reports only
Software toolchain	RISC-V GCC (riscv32 cross-compiler)
Reference model	Spike or a custom Python model
Version control and CI	Git, GitHub, GitHub Actions
Documentation	Markdown, block diagrams, waveform screenshots

5. Work Plan and Timeline

Estimated duration: **10 weeks**, at a pace compatible with my academic workload. The schedule is flexible and can be adjusted based on supervisor feedback.

Phase	Weeks	Work	Deliverable
1. Foundations	1-2	VHDL refresher, toolchain setup (GHDL, GTKWave, cocotb), simple blocks and FSMs, first testbenches	Working simulation environment and sample modules
2. Single-cycle CPU	3-4	ALU, register file, decoder, memories, single-cycle datapath for RV32I	Single-cycle CPU passing instruction-level tests
3. Pipelining	5-6	Split into five stages, forwarding unit, hazard detection, branch handling	Pipelined CPU passing all instruction tests
4. Program-level verification	7-8	Run C programs, compare traces against the reference model, add coverage and assertions, set up CI	Automated regression suite with CI

Phase	Weeks	Work	Deliverable
5. Synthesis and analysis	9	Synthesis, timing and resource reports, CPI and performance comparison versus single-cycle	Synthesis report and performance analysis
6. Documentation	10	README, architecture diagrams, final report, demo of waveforms and test runs	Final repository and report

Stretch goals (cache, UART, custom instruction, M extension) are scheduled only if phases 1 to 5 finish ahead of plan.

6. Expected Deliverables

- Complete VHDL source code of the CPU with clear module structure.
- Testbench suite and Python (cocotb) verification environment.
- Collection of assembly and C test programs.
- Automated CI pipeline running the full regression.
- Synthesis, timing and resource report for a target FPGA family.
- Technical documentation with block diagrams and waveforms.
- Final report summarizing design decisions, results and lessons learned.

7. Success Metrics

- 100% of the RV32I instructions pass their tests.
- All test C programs produce results identical to the reference model.
- Hazards (data, load-use, control) handled with no functional errors in the regression.
- Reported maximum frequency, resource usage and CPI, with a comparison to the single-cycle baseline.
- Reproducible setup: a new user can clone the repository and run the full test suite with one command.

8. Risks and Mitigation

Risk	Mitigation
Learning curve of VHDL and hardware debugging	Start with small modules and testbenches; simulate before integrating
Pipeline hazard bugs that are hard to trace	Reference-model trace comparison and targeted hazard test programs
Toolchain setup issues on my machine	Use a Linux VM or container; document the setup steps early
Schedule overrun due to coursework	Fixed core scope, stretch goals optional, weekly progress check
No physical board for hardware validation	Rely on simulation and synthesis reports; board demo kept as optional extension

9. Expected Benefits

- Practical mastery of RTL design, simulation-based verification and timing analysis.
- A concrete, measurable engineering result that complements my embedded and IoT experience.
- A well-documented, reproducible project that can be reused for my end-of-studies internship applications.

10. Questions for the Supervisor

1. Is the scope (RV32I, 5-stage pipeline, 10 weeks) appropriate, or should it be reduced or extended?
2. Do you recommend a specific verification methodology or tool I should include (for example assertions or formal checks)?
3. Would you prefer the project to include a particular extension (cache, M extension, custom accelerator)?
4. Is there a target FPGA family or industry use case you would like me to align the synthesis step with?
5. What reporting format and milestone reviews do you expect?